Compiler design and programming languages

Lecture 01

1st 2019-2020

The Course covers:

- Lexical Analysis
- Syntax Analysis
- Semantic Analysis
- Runtime environments
- Code Generation
- Code Optimization

Pre-requisite courses

- **Strong** programming background in Python, C, C++ or Java.
- Formal Language (NFAs, DFAs, CFG)

3

Resources

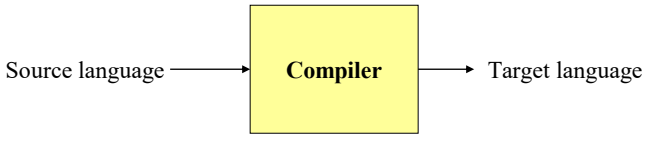
- Textbooks:
  - *Compilers: Principles, Techniques and Tools*, Aho, Lam, Sethi & Ullman, 2007 (required)
  - *lex & yacc*, Levine et. al.
- Handouts
- Sample code for Lex/YACC (C, C++, Java)

4

Lecture 1: Introduction to Language Processing & Lexical Analysis

What is a compiler?

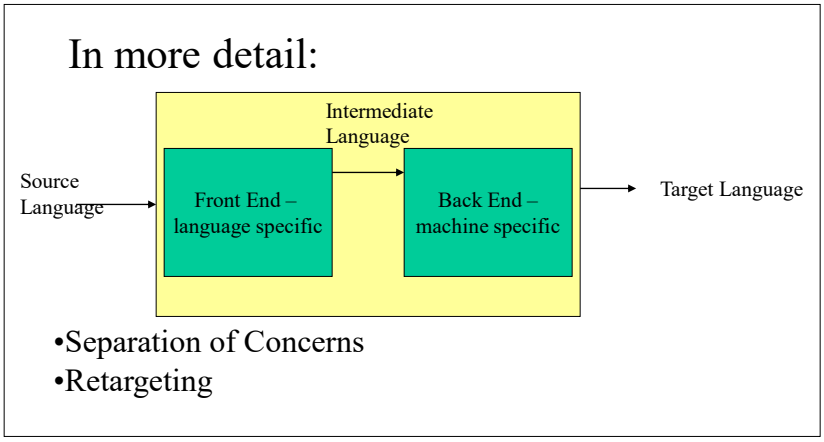
A program that reads a program written in one language and translates it into another language.



Traditionally, compilers go from high-level languages to low-level languages.

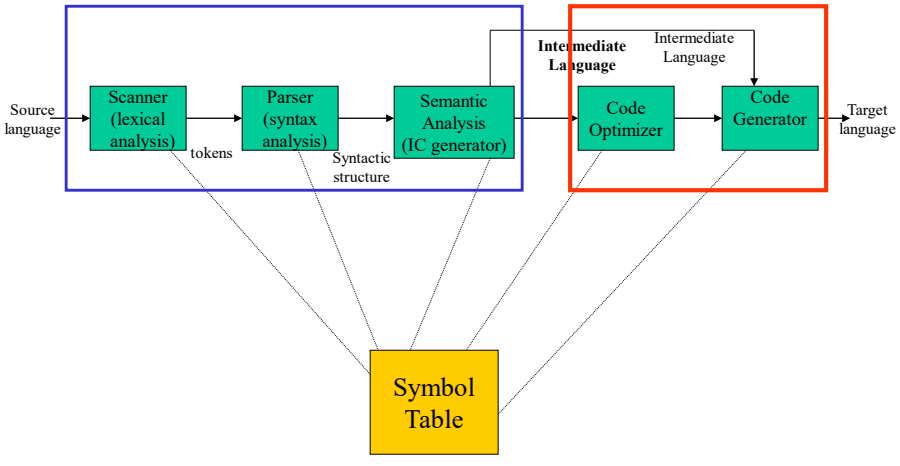
6

Compiler Architecture



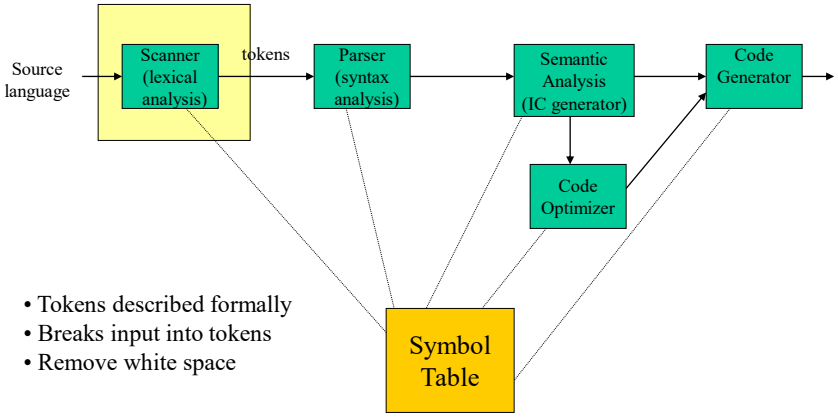
7

Compiler Architecture



8

Lexical Analysis - Scanning



- Tokens described formally
- Breaks input into tokens
- Remove white space

9

Input: result = a + b * c / d

- Tokens:

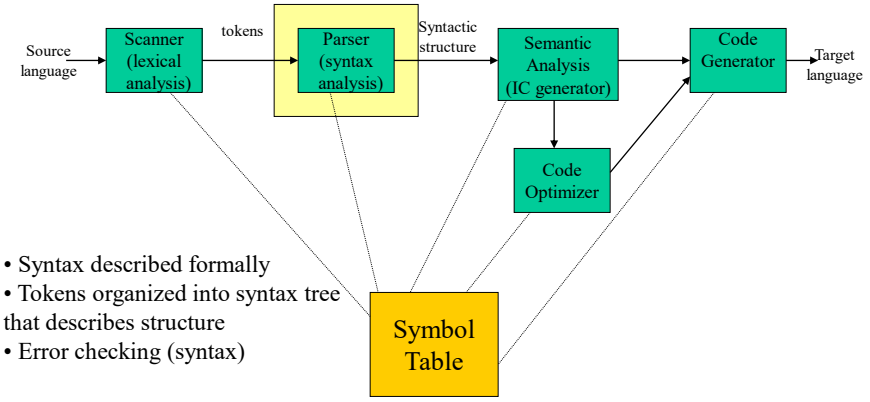
‘result’, ‘=‘, ‘a’, ‘+’, ‘b’, ‘*’, ‘c’, ‘/’, ‘d’

identifiers

operators

10

Static Analysis - Parsing

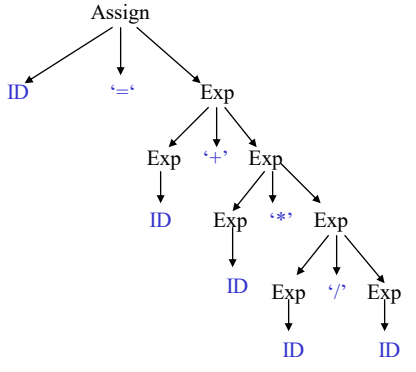


- Syntax described formally
- Tokens organized into syntax tree that describes structure
- Error checking (syntax)

11

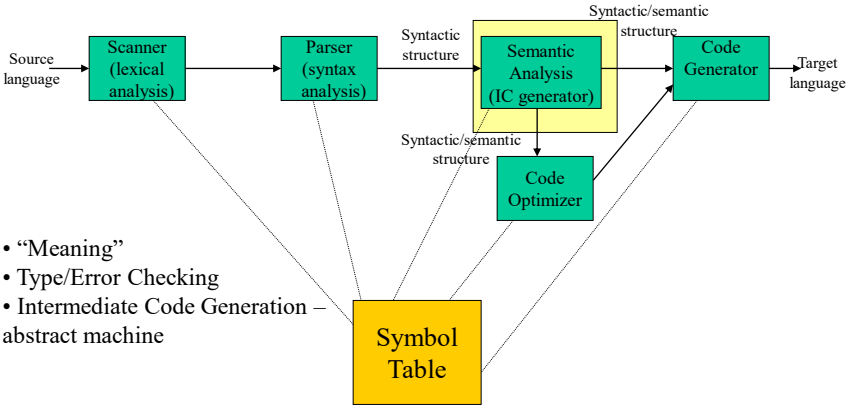
Input: result = a + b * c / d

```
Exp    ::= Exp '+' Exp
         |  Exp '-' Exp
         |  Exp '*' Exp
         |  Exp '/' Exp
         |  ID
Assign ::=  ID '=' Exp
```



12

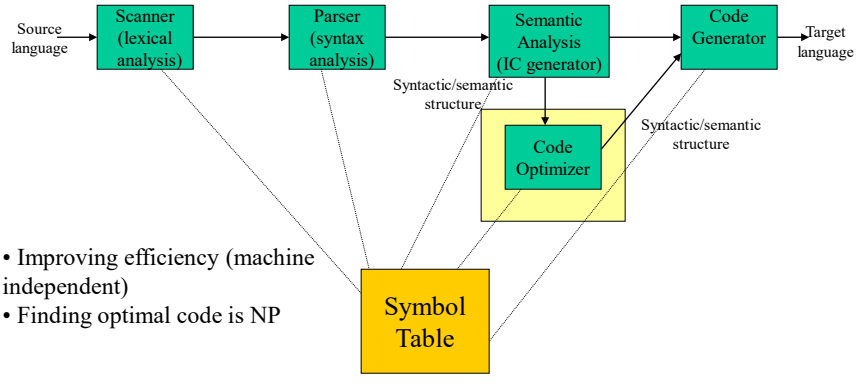
Semantic Analysis



- "Meaning"
- Type/Error Checking
- Intermediate Code Generation – abstract machine

13

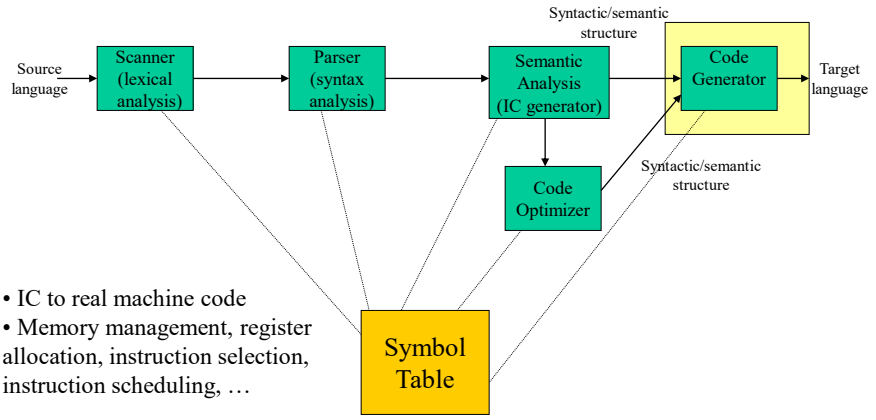
Optimization



- Improving efficiency (machine independent)
- Finding optimal code is NP

14

Code Generation



- IC to real machine code
- Memory management, register allocation, instruction selection, instruction scheduling, …

15

Issues Driving Compiler Design

- Correctness
- Speed (runtime and compile time)
  - Degrees of optimization
  - Multiple passes
- Space
- Feedback to user
- Debugging

16

Related to Compilers

- Interpreters (direct execution)
- Assemblers
- Preprocessors
- Text formatters (non-WYSIWYG)
- Analysis tools

17

Why study compilers?

- Bring together:
  - Data structures & Algorithms
  - Formal Languages
  - Computer Architecture
- Influence:
  - Language Design
  - Architecture (influence is bi-directional)
- Techniques used influence other areas (program analysis, testing, …)

18

Review of Formal Languages

- Regular expressions, NFA, DFA
- Translating between formalisms
- Using these formalisms

19

What is a language?

- **Alphabet** – finite character set (Σ)
- **String** – finite sequence of characters – can be ε, the empty string (Some texts use λ as the empty string)
- **Language** – possibly infinite set of strings over some alphabet – can be { }, the empty language.

20

Suppose Σ = {a,b,c}. Some languages over Σ could be:

- {aa,ab,ac,bb,bc,cc}
- {ab,abc,abcc,abccc,. . .}
- { ε }
- { }
- {a,b,c,ε}
- …

21

Why do we care about Regular Languages?

- Formally describe tokens in the language
  - Regular Expressions
  - NFA
  - DFA
- Regular Expressions ➔ finite automata
- Tools assist in the process

22

Regular Expressions

The **regular expressions** over finite Σ are the strings over the alphabet Σ + {), (, |, * } such that:

1. { } (empty set) is a regular expression for the empty set
2. ε is a regular expression denoting { ε }
3. *a* is a regular expression denoting set { *a* } for any *a* in Σ

23

Regular Expressions

4. If P and Q are regular expressions over Σ, then so are:
   - **P | Q (union)**
     If P denotes the set {*a*,…,*e*}, Q denotes the set {*0*,…,*9*} then P | Q denotes the set {*a*,…,*e*,*0*,…,*9*}
   - **PQ (concatenation)**
     If P denotes the set {*a*,…,*e*}, Q denotes the set {*0*,…,*9*} then PQ denotes the set {*a0*,…,*e0*,*a1*,…,*e9*}
   - **Q* (closure)**
     If Q denotes the set {*0*,…,*9*} then Q* denotes the set {*0*,…,*9*,*00*,…*99*,…}

24

Examples

If $\Sigma = \{a,b\}$

- (a | b)(a | b)
- (a | b)*b
- a*b*a*
- a*a (also known as a+)
- (ab*)|(a*b)

25

Nondeterministic Finite Automata

A **nondeterministic finite automaton** (NFA) is a mathematical model that consists of

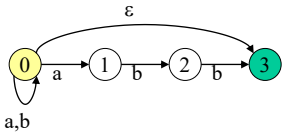
1. A set of states S
2. A set of input symbols Σ
3. A transition function that maps state/symbol pairs to a set of states:
   S x {Σ + ε} → set of S
4. A special state s_0 called the start state
5. A set of states F (subset of S) of final states

INPUT: string

OUTPUT: yes or no

26

Example NFA



Transition Table:

STATE	a	b	ε
0	0,1	0	3
1		2	
2		3	
3			

$S = \{0,1,2,3\}$
$S_0 = 0$
$\Sigma = \{a,b\}$
$F = \{3\}$

27

NFA Execution

An NFA says 'yes' for an input string if there is some path from the start state to some final state where all input has been processed.

```
NFA(int s0, int input_element) {
   if (all input processed and s0 is a final state) return Yes;
   if (all input processed and s0 is not a final state) return No;

   for all states s1 where transition(s0,table[input_element]) = s1
        if (NFA(s1,input_element+1) = = Yes) return Yes;

   for all states s1 where transition(s0,ε) = s1
        if (NFA(s1,input_element) = = Yes) return Yes;
    return No;
}
```

Uses backtracking to search all possible paths

28

Deterministic Finite Automata

A **deterministic finite automaton** (DFA) is a mathematical model that consists of

1. A set of states S
2. A set of input symbols Σ
3. A transition function that maps state/symbol pairs to a state:
   S x Σ → S
4. A special state s_0 called the start state
5. A set of states F (subset of S) of final states

INPUT: string

OUTPUT: yes or no

29

DFA Execution

```
DFA(int start_state) {
   state current = start_state;
   input_element = next_token();
   while (input to be processed) {
       current =
              transition(current,table[input_element])
       if current is an error state return No;
       input_element = next_token();
    }
    if current is a final state return Yes;
    else return No;
}
```

30

Regular Languages

1. There is an algorithm for converting any RE into an NFA.
2. There is an algorithm for converting any NFA to a DFA.
3. There is an algorithm for converting any DFA to a RE.

These facts tell us that REs, NFAs and DFAs have equivalent expressive power. All three describe the class of regular languages.

31

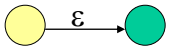
Converting Regular Expressions to NFAs

The **regular expressions** over finite Σ are the strings over the alphabet Σ + {), (, |, *} such that:

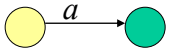
- { } (empty set) is a regular expression for the empty set



- Empty string ε is a regular expression denoting { ε }



- *a* is a regular expression denoting {*a* } for any *a* in Σ

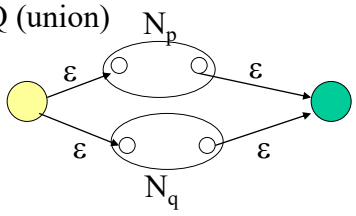


32

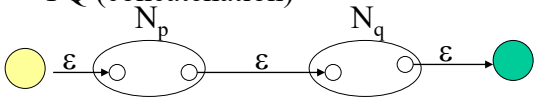
Converting Regular Expressions to NFAs

If P and Q are regular expressions with NFAs N_p, N_q:

P | Q (union)



PQ (concatenation)

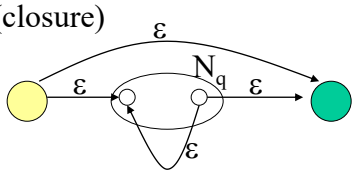


33

Converting Regular Expressions to NFAs

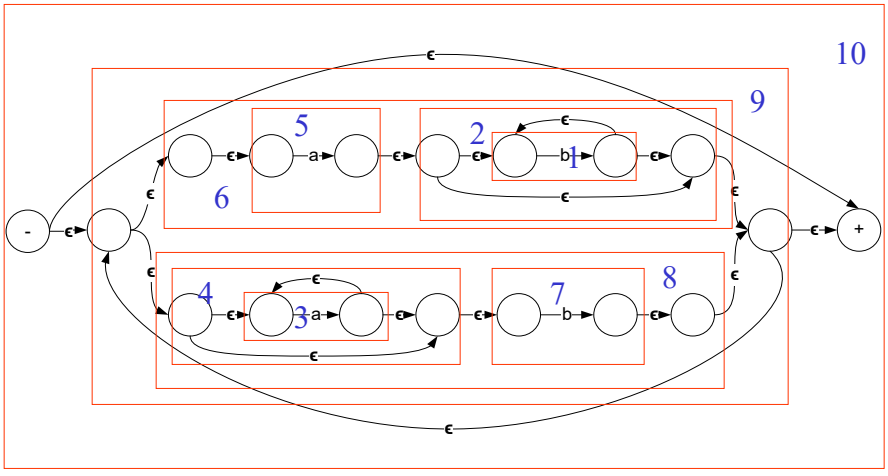
If Q is a regular expression with NFA N_q:

Q* (closure)



34

Example (ab* | a*b)*



35

Converting NFAs to DFAs

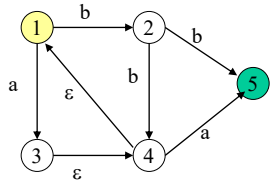
- **Idea**: Each state in the new DFA will correspond to some set of states from the NFA. The DFA will be in state $\{s_0,s_1,\ldots\}$ after input if the NFA could be in *any* of these states for the same input.

- **Input**: NFA N with state set S_N, alphabet Σ, start state s_N, final states F_N, transition function T_N: $S_N \times \Sigma + \{\varepsilon\} \rightarrow$ set of S_N
- **Output**: DFA D with state set S_D, alphabet Σ, start state $s_D = \varepsilon\text{-closure}(s_N)$, final states F_D, transition function T_D: $S_D \times \Sigma \rightarrow S_D$

36

ε-closure()

Defn: ε-closure(T) = T + all NFA states reachable from any state in T using only ε transitions.



ε-closure({1,2,5}) = {1,2,5}
ε-closure({4}) = {1,4}
ε-closure({3}) = {1,3,4}
ε-closure({3,5}) = {1,3,4,5}

37

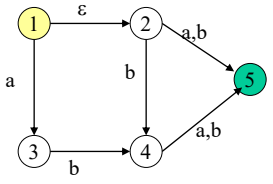
Algorithm: Subset Construction

s_D = ε-closure(s_N) -- create start state for DFA
S_D = {s_D} (unmarked)
while there is some unmarked state **R** in S_D
    mark state **R**
    for all *a* in Σ do
        s = ε-closure(T_N(**R**,*a*));
        if s not already in S_D then add it (unmarked)
        T_D(**R**,*a*) = s;
    end for
end while
F_D = any element of S_D that contains a state in F_N

38

Example 1: Subset Construction

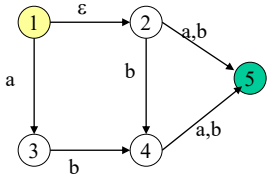
NFA



39

Example 1: Subset Construction

NFA



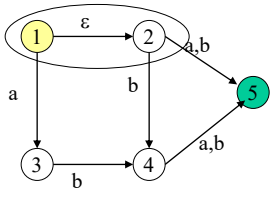
1,2

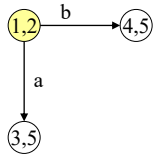
	a	b
{1,2}		

40

Example 1: Subset Construction

NFA



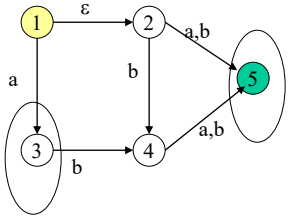


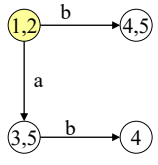
	a	b
{1,2}	{3,5}	{4,5}
{3,5}		
{4,5}		

41

Example 1: Subset Construction

NFA



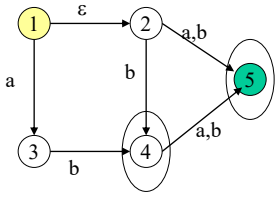


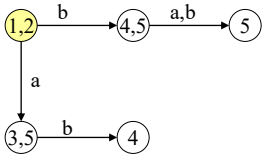
	a	b
{1,2}	{3,5}	{4,5}
{3,5}	-	{4}
{4,5}		
{4}		

42

Example 1: Subset Construction

NFA



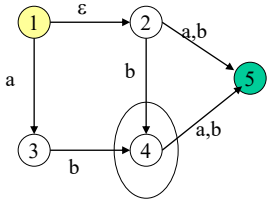


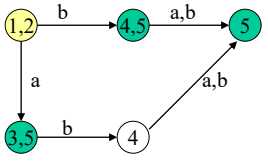
	a	b
{1,2}	{3,5}	{4,5}
{3,5}	-	{4}
{4,5}	{5}	{5}
{4}		
{5}		

43

Example 1: Subset Construction

NFA





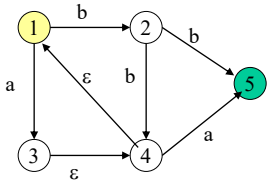
All final states since the NFA final state is included

	a	b
{1,2}	{3,5}	{4,5}
{3,5}	-	{4}
{4,5}	{5}	{5}
{4}	{5}	{5}
{5}	-	-

44

Example 2: Subset Construction

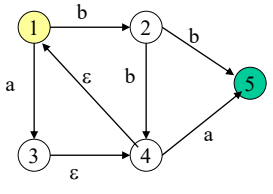
NFA



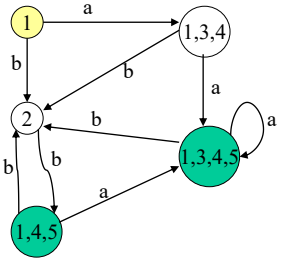
45

Example 2: Subset Construction

NFA



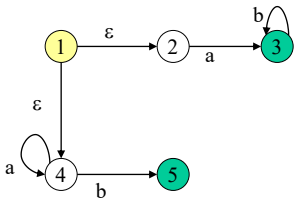
DFA



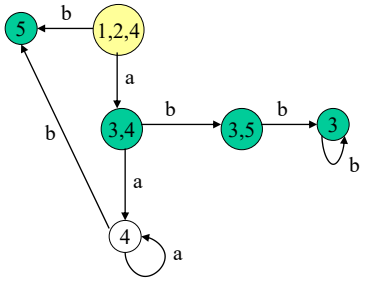
46

Example 3: Subset Construction

NFA



DFA



47

Converting DFAs to REs

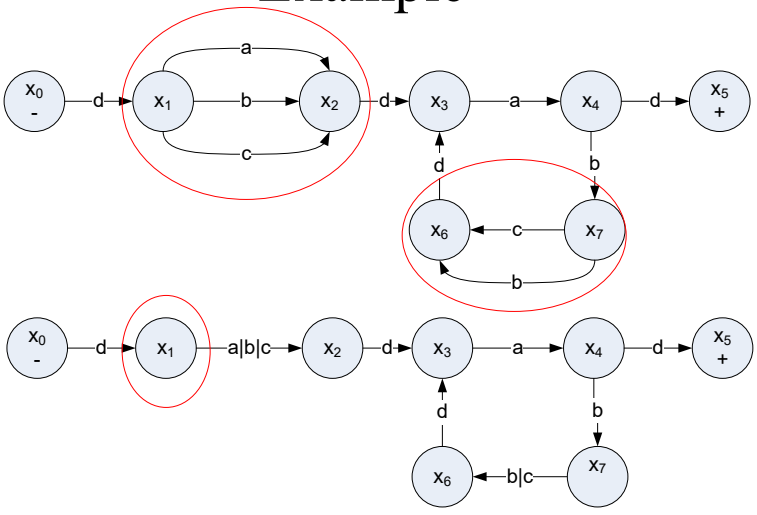
1. Add (if necessary) a unique start state without incoming edges and a unique final state without outgoing edges..
2. Combine parallel links by alternation

For each state that is not a start state or a final state, repeat steps 2 and 3.

3. Do a bypass and state elimination operation.
4. Combine edges that have the same starting and ending state.
5. Combine the edges between the start state and the final state. The label on the only remaining edge is the regular expression result. If there is none, the regular expression is ϕ.

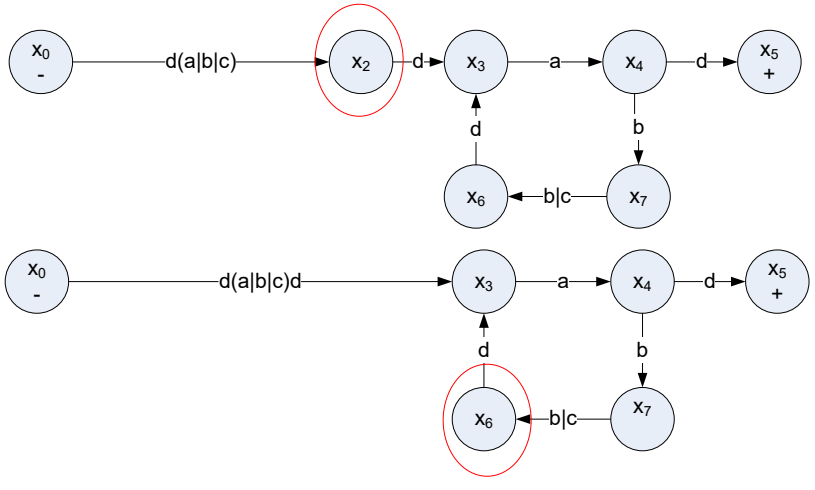
48

Example



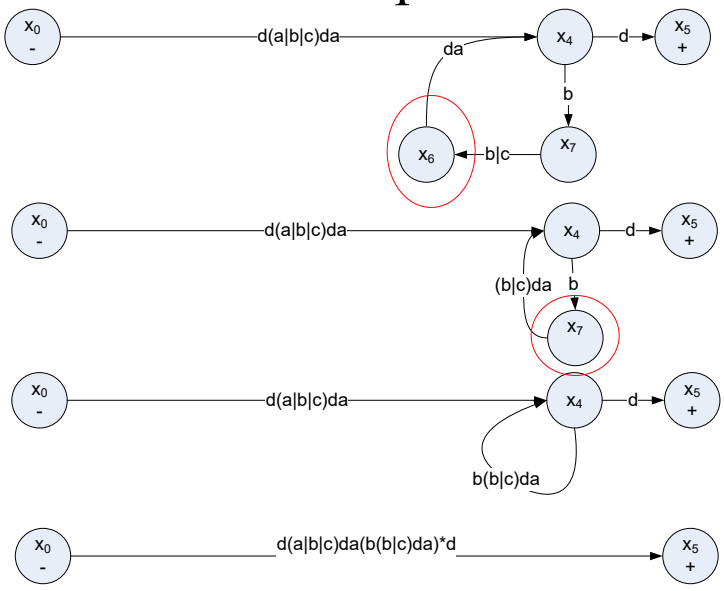
49

Example



50

Example



51

Describing Regular Languages

- Generate ***all*** strings in the language
- Generate ***only*** strings in the language

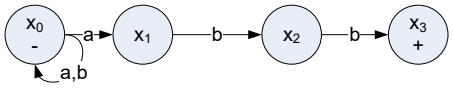
Try the following:

- Strings of {*a*,*b*} that end with '*abb*'
- Strings of {*a*,*b*} that don't end with '*abb*'
- Strings of {*a*,*b*} where every *a* is followed by at least one *b*

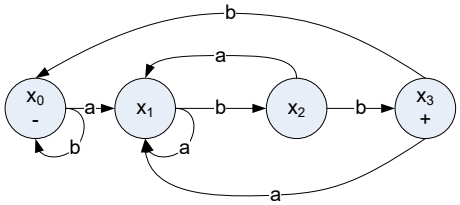
52

Strings of (a|b)* that end in abb

re: (a|b)*abb



NFA

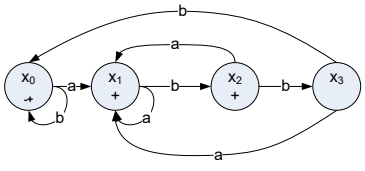


DFA

53

Strings of (a|b)* that don't end in abb

re: ??



DFA/NFA

54

Suggestions for writing NFA/DFA/RE

- Typically, one of these formalisms is more natural for the problem. Start with that and convert if necessary.
- In NFA/DFAs, each state typically captures some partial solution
- Be sure that you include all relevant edges (ask – does every state have an outgoing transition for all alphabet symbols?)

55

Non-Regular Languages

Not all languages are regular"

- The language *ww* where *w*=(a|b)*

Non-regular languages cannot be described using REs, NFAs and DFAs.

56